



UNIVERSIDADE ESTADUAL DE MARINGÁ

Centro de Tecnologia - CTC
Departamento de Informática - DIN

ARQUITETURA E ORGANIZAÇÃO DE COMPUTADORES
BACHARELADO EM CIÊNCIA DA COMPUTAÇÃO – 1º sem/2025
Prof. Rodrigo Calvo

AVALIAÇÃO 3 - PROJETO SIMULAÇÃO DO CICLO DE INSTRUÇÃO

Objetivo: Desenvolver um programa que simule o comportamento dos registradores de um processador em um ciclo de instrução ao executar uma sequência de comandos de linguagem de máquina. Os registradores devem armazenar dados temporariamente para que tais comandos sejam executados.

Descrição: O simulador desenvolvido deve executar operações básicas de linguagem de máquina, como adição, subtração, divisão, multiplicação, transferência de dados, e desvio de fluxo. Todas as instruções do algoritmo e os dados contidos nele devem estar armazenados em memória RAM. As instruções devem ser armazenadas sequencialmente na RAM e em local distinto do armazenamento de dados. A seguir, é fornecida a especificação da máquina simulada:

Memória

- Tamanho: 256 posições
- Endereçamento: hexadecimal (0x00 a 0xFF)
- Cada posição armazena um valor inteiro ou uma instrução

Registradores

Uso geral: A, B

Especiais:

- PC: *Program Counter*
- IR: *Instruction Register*
- MAR: *Memory Address Register*
- MBR: *Memory Buffer Register*
- AC: *Acumulador*
- M: *Multiplicador*
- R: *Resto da divisão*
- C: *Carry* (0 ou 1)
- N: 0 (positivo) ou 1 (negativo)
- Z: 0 (diferente de zero) ou 1 (zero)

Ciclo de Instrução

1. $MAR \leftarrow PC$
2. $MBR \leftarrow MEM[MAR]$
3. $IR \leftarrow MBR$
4. $PC \leftarrow PC + 1$
5. Executar instrução

Conjunto de Instruções: conjunto de instruções do computador IAS (visto em aula)

O programa (simulador) deve receber como entrada um arquivo texto com os dados e a sequência de comandos. Após a leitura deste arquivo, o programa em linguagem de máquina e os dados utilizados nele estão representados em uma memória (ou vetor no programa). Este vetor deve simular o armazenamento de dados na RAM. Cada posição do vetor é endereçada por um valor no sistema hexadecimal. O mapeamento entre as posições do vetor da memória ocorre da seguinte maneira: a primeira posição do vetor equivale ao endereço 0x00, a segunda equivale ao endereço 0x01 e assim sucessivamente. Por exemplo, a décima primeira posição do vetor equivale ao endereço 0x0A da RAM. Um exemplo de arquivo texto de entrada é mostrado a seguir:

```
# Dados
3
4
10

# Endereço inicial da primeira instrução
0xA0

LOAD    A, 0x05
ADD     A, 0x02
MOV     B, A
MULT    B, 0x06
ADD     B, 5
STORE   B, 5
```

A execução do algoritmo deverá ocorrer de forma sequencial, com uma instrução executada por vez. Após cada execução, o simulador deverá exibir o estado atualizado de todos os registradores, incluindo os de propósito geral e os registradores especiais.

O simulador deverá executar o algoritmo de ordenação por Seleção (Selecion Sort), fornecido neste enunciado, e um segundo algoritmo de livre escolha, destinado à resolução de qualquer outro problema.

Considere:

- Todos os dados envolvidos nos comandos devem ser inteiros;
- O conjunto de instruções do computador IAS, assim como o tamanho das palavras e instruções.
- Endereçamento Imediato e Direto (permitidos outros modos de endereçamento);
- Não há chamada de procedimento (permitido);
- Não é necessário salvar o contexto do processador para restaurá-lo posteriormente (permitido se houver chamada de procedimento);
- Não há interrupções;

Como o objetivo deste projeto é simular o comportamento dos registradores de um processador, o conteúdo de todos os registradores deve ser impresso em tela para cada comando da sequência. Para melhor visualizar a saída do programa, controle o fluxo de execução do algoritmo tecendo <ENTER> entre a execução de dois comandos.

Antes de executar a primeira instrução, o conteúdo dos registrados deve ser exibido em tela. Lembre-se que neste instante somente o registrador PC possuirá conteúdo válido.

Relatório Técnico

Além da implementação do simulador, deve ser entregue um relatório técnico contendo:

- **Introdução:** contexto, objetivos e visão geral do simulador;
- **Desenvolvimento:**
 - Descrição da implementação;
 - Organização da memória e dos registradores;
 - Conjunto de instruções utilizado (indicando se houve modificações);
 - Principais decisões de projeto;
 - Linguagem de programação adotada;
- **Execução do simulador:**
 - Instruções para compilação e execução;
 - Comandos utilizados;
 - Dependências necessárias;
- **Resultados:**
 - Testes realizados;
 - Exemplos de execução;
 - Imagens (prints) do simulador em funcionamento, evidenciando os estados dos registradores;

- **Conclusão:**
 - Análise dos resultados;
 - Dificuldades encontradas;
 - Possíveis melhorias futuras;

O relatório deve ser claro e detalhado o suficiente para permitir que outro usuário compreenda e execute o simulador sem auxílio adicional.

- Serão avaliados:
 - ✓ Vazamento de memória, referência a valores de variáveis não inicializadas;
 - ✓ Comentários em trechos de código;
 - ✓ Envie junto com o projeto um arquivo (readme.txt, os passos necessários executar os programas (comandos e parâmetros de entrada).
- Implemente usando a linguagem C ou Python;
- Os trabalhos deverão ser feitos até em trio;
- Serão verificados trabalhos nos quais haja quaisquer tipos de cópia ou plágio.
- O algoritmo escolhido deve ser diferente de qualquer outro grupo.

Modo de entrega: Fazer upload no Classroom. A primeira linha do código do arquivo, deve haver o nome completo e RA dos alunos em comentário.

Prazo de entrega: 06/07/2026

Selection Sort

# Dados 5 # N 1 # constante 1 0 # i 0 # j 0 # min 0 # temp 0x110 # base do vetor 10 3 7 2 8 # Ponteiros auxiliares 0 # addr_i 0 # addr_j 0 # addr_min # Endereço inicial 0xA0 # i = 0 LOAD M(0x101) SUB M(0x101) STOR M(0x102) # LOOP_I LOAD M(0x102) SUB M(0x100) JUMP+ M(0xF0) # min = i LOAD M(0x102) STOR M(0x104) # j = i + 1 LOAD M(0x102) ADD M(0x101) STOR M(0x103) # LOOP_J LOAD M(0x103) SUB M(0x100) JUMP+ M(0xD0)	# addr_j = base + j LOAD M(0x106) ADD M(0x103) STOR M(0x120) # addr_min = base + min LOAD M(0x106) ADD M(0x104) STOR M(0x121) # A[j] LOADI M(0x120) STOR M(0x130) # A[min] LOADI M(0x121) STOR M(0x131) # comparação LOAD M(0x130) SUB M(0x131) JUMP+ M(0xC0) # min = j LOAD M(0x103) STOR M(0x104) # NEXT_J LOAD M(0x103) ADD M(0x101) STOR M(0x103) JUMP M(0xAA) # TROCA # addr_i = base + i LOAD M(0x106) ADD M(0x102) STOR M(0x122) # addr_min = base + min LOAD M(0x106) ADD M(0x104) STOR M(0x123)	# temp = A[i] LOADI M(0x122) STOR M(0x105) # A[i] = A[min] LOADI M(0x123) STORI M(0x122) # A[min] = temp LOAD M(0x105) STORI M(0x123) # i++ LOAD M(0x102) ADD M(0x101) STOR M(0x102) JUMP M(0xA3) # FIM
--	--	---

Mapa de Memória

0x100	→ N
0x101	→ l
0x102	→ i
0x103	→ j
0x104	→ min
0x105	→ temp
0x106	→ base do vetor
0x110...	→ vetor
0x120	→ addr_j
0x121	→ addr_min
0x122	→ addr_i
0x123	→ addr_min (troca)
0x130	→ A[j]
0x131	→ A[min]